

QLoRA Domain Adaptation for Structured Food Extraction

Fine-tuning a 3B instruct model to JSON on a single 8 GB GPU

CSCI E-222: Foundations of Large Language Models

Niraj Thapaliya

May 16, 2026

Abstract

This project investigates whether a small, modern instruct model can be adapted to a narrow structured-extraction task, ie turning unstructured English food/drink text into a valid JSON object with a fixed four-key schema using parameter-efficient fine-tuning on a single consumer GPU. The base model is “Qwen/Qwen2.5-3B-Instruct”, loaded in 4-bit NF4 quantization. We attach LoRA adapters (rank 16) to all linear layers, train on a seeded 70:15:15 split of “mrdbourke/FoodExtract-1k” (1,420 rows), and evaluate against the un-tuned base on seven metrics that score JSON validity, schema correctness, per-field F1, and exact match.

The whole pipeline, including exploratory analysis, label normalization, prompt formatting, completion-only loss masking, 4-bit base loading, adapter training, and before/after evaluation, runs end-to-end in a single Jupyter notebook on an NVIDIA RTX 3060 Ti (8 GB). Trainable parameters are 29.9 M of 3.12 B (0.96%); peak VRAM during training stays well under 8 GB. Training takes approximately ~45 minutes seconds on the 3060 Ti and reaches a final training loss of 0.089.

After fine-tuning, exact-match on the 213-example test split rises from 0.258 (baseline) to 0.624 (fine-tuned), and schema correctness rises from 0.925 to 1.000. Per-field F1 improvements are reported in below. The conclusion is that QLoRA on a 3B base is a practical, fully-on-device recipe for domain adaptation when the target task has a tight output schema and a modest dataset.

1. Introduction and Problem Statement

Large pretrained language models excel at open-ended generation but are routinely asked to produce strict, machine-readable output in real systems: tool calls, function arguments, database rows, structured logs. Even strong instruct models drift off the schema or wrap valid JSON in prose, code fences, or apologies. Two strategies dominate in practice: (a) keep the model general and constrain output at decoding time, or (b) cheaply adapt the model to the target schema via parameter-efficient fine-tuning (PEFT). This project takes the second route on a single consumer GPU.

The task is narrow and concrete. Given an unstructured English caption of a food or drink scene, produce one JSON object with exactly four keys:

```
{
  "is_food_or_drink": true,
  "tags": ["fi", "fa"],
  "food_items": ["cheese-stuffed peppers", "cherry tomato halves"],
  "drink_items": []
}
```

The contract is tight enough to measure cleanly: JSON parses or it does not, keys are present or not, item sets agree with the gold set or not. There is no room for the model to be "basically right", a missing closing brace is a miss even if the output is 99% correct.

The motivation is partly pedagogical: this is also an investigation of how small a GPU and how short a training run can usefully fine-tune a modern instruct model. The target hardware is a single NVIDIA RTX 3060 Ti with 8 GB of VRAM. Every design decision in the pipeline, such as 4-bit quantization, LoRA rank, gradient checkpointing, paged 8-bit Adam, batch size 1 with gradient accumulation, and a 1024-token context, is in service of fitting cleanly inside that budget.

2. Data Description and Preprocessing

2.1 Dataset

We use “mrdbourke/FoodExtract-1k” from the Hugging Face Hub. The dataset ships as a single train split with 1,420 rows. Each row pairs a long English input (sequence: typically an auto-generated image caption) with a “gpt-oss-120b-label” string that is intended to be the JSON target. Inputs cover both food/drink scenes and unrelated content (people, animals, objects), so the model has to learn both when to extract items and when to return an empty object.

FoodExtract-1k is small (about 1 MB) and balanced — `class_label` is exactly 50% food, 50% not_food which makes it well-suited to fitting an adapter on a small GPU without resampling. A larger version of the same dataset (FoodExtract-135k) exists but is not used here; the project's research question is about what a modest setup can accomplish.

2.2 EDA and label cleaning

The “gpt-oss-120b-label” column does not parse as JSON. It is the Python repr of a dict (`dict.to_string()`). It has single quotes and capitalized True/False, so `json.loads` fails on every row. We use `ast.literal_eval` to recover all 1,420 rows. The recovered schema is uniform: every row has exactly the four keys (`is_food_or_drink`, `tags`, `food_items`, `drink_items`).

We found another thing to clean up: `is_food_or_drink` is usually a Python bool, but in 53 rows it is the string 'true' or 'false'. The preprocessor coerces these with `v.strip().lower() == 'true'` before re-serializing. The output target is canonical JSON with double-quoted keys, true/false, and Unicode preserved.

2.3 Split

There is no shipped validation/test split, so we cut a seeded (SEED=222) 70:15:15 split with Hugging Face's `train_test_split`, resulting in 994 train, 213 validation, 213 test items. The split is seeded so the reported numbers are reproducible. Each split has 100% valid-JSON targets after normalization, and class balance is preserved within $\pm 3\%$ of the source distribution.

2.4 Length analysis and MAX_SEQ_LEN

We tokenize the full system+user+assistant chat-format sequence with the base model's tokenizer to pick MAX_SEQ_LEN empirically. The 99th-percentile combined length is well under 1,024 tokens and the max is also under 1,024, which means 1,024 covers 100% of training rows. A larger cap would waste compute on padding and a smaller cap would silently truncate a few targets.

3. Models and Methods

3.1 Base model

The base model is “Qwen/Qwen2.5-3B-Instruct”, which has 3.1 B parameters, a modern decoder-only architecture with grouped-query attention and a long-context (32k) tokenizer. It is small enough to load in 4-bit on a 3060 Ti with room for gradients and activations, large enough to follow JSON instructions out of the box, and permissively licensed (Apache-2.0). It already “knows” the JSON output format well enough to produce a useful baseline.

3.2 QLoRA in one paragraph

QLoRA freezes the base model in 4-bit NF4 quantization and trains small low-rank adapter matrices on top in higher precision. The base weights are not updated; only the adapters are. We use NF4 quantization with double quantization, bfloat16 compute, and LoRA adapters of rank 16, dropout 0.05, attached to every linear layer in the transformer block (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj). This follows the QLoRA paper's finding that broadening the LoRA target set is more important than rank for closing the gap to full fine-tuning.

At the end, we drop to 29,933,568 trainable parameters from 3,115,872,256 total (0.9607% of the model). The 4-bit base alone fits in ~2.0 GB of VRAM.

3.3 Prompt format and loss masking

Each example is formatted with the tokenizer's chat template (system, user, assistant) so it matches the format “Qwen2.5-Instruct” was trained on. The assistant turn contains the canonical JSON target. During training we mask the prompt tokens (system + user) with `label_id = -100` so the loss is computed only on the JSON completion. This prevents the model from "learning" to copy the system instruction or the input text and focuses optimization on the contract.

The system prompt is fixed across all examples and specifies the four-key schema with field types, so the model never has to infer the contract at inference time.

4. Implementation Details

4.1 Environment

The exact environment is pinned with uv in `pyproject.toml` + `uv.lock`. Key versions:

Package	Version	Role
torch	2.11.0 + CUDA 13.0	tensor + autograd
transformers	5.5.4	Trainer, AutoModelForCausalLM, tokenizer
peft	0.19.0	LoRA adapters, <code>prepare_model_for_kbit_training</code>
bitsandbytes	0.49.2	4-bit NF4 quantization, paged 8-bit Adam
accelerate	1.13+	device map, gradient checkpointing
datasets	4.8+	HF Hub dataset loading + split

4.2 Hardware

My 2021 PC has an NVIDIA RTX 3060 Ti (8 GB VRAM). After 4-bit base load, LoRA adapter attachment, gradient checkpointing, and paged 8-bit Adam state, the peak VRAM during training stays well under the 8 GB ceiling, leaving some wiggle room for longer sequences or higher rank if desired.

4.3 Training hyperparameters

Hyperparameter	Value	Notes
Learning rate	2e-4	QLoRA paper recommendation for instruct bases
LR schedule	cosine	with 10 warmup steps
Optimizer	paged_adamw_8bit	8-bit Adam state, pages to CPU if needed
Per-device batch size	1	
Gradient accumulation	16	effective batch size = 16
Epochs	3	3 was sufficient
Precision	bf16	Compact float16 representation
Gradient checkpointing	on	
Seed	222	

4.4 How to run

The code is in the Jupyter notebook, `food_extract_qlora.ipynb`. To run, download it to an empty directory, then run these bash commands:

```
# make sure you have `uv` installed
uv python pin 3.14
uv init --bare
uv add transformers peft bitsandbytes accelerate datasets evaluate matplotlib
pandas numpy
uv add --dev jupyter-lab
uv run jupyter lab
```

The fine-tuned adapter is saved to “`qlora-foodextract-adapter/`”.

5. Experiments and Results

5.1 Evaluation harness

We evaluate on the held out 213 row test split. Each metric is computed over all 213 examples: a malformed output counts as a miss for every downstream metric, so there is no "partial credit" hidden by JSON failures.

- `json_valid` - the model's output parses as JSON.
- `schema_ok` - parses and has exactly the 4 expected keys with the right types.
- `is_food_acc` - accuracy on the `is_food_or_drink` boolean.
- `tags_f1/food_items_f1/drink_items_f1` - case-insensitive set-level F1 per list field.
- `exact_match` - prediction equals gold JSON exactly after canonical key ordering.

5.2 Baseline vs fine-tuned (test split, 213 rows)

Metric	Baseline	Fine-tuned	Difference
<code>json_valid</code>	0.972	1.000	+0.028
<code>schema_ok</code>	0.925	1.000	+0.075
<code>is_food_acc</code>	0.798	0.972	+0.174
<code>tags_f1</code>	0.305	0.943	+0.638
<code>food_items_f1</code>	0.665	0.861	+0.196
<code>drink_items_f1</code>	0.875	0.949	+0.074
<code>exact_match</code>	0.258	0.624	+0.366

The clearest improvement is on `exact_match` (from 0.258 to 0.624, +0.366) and `tags_f1` (0.305 to 0.943, +0.638): both are sensitive to whether the model emits the dataset-specific short tag codes rather than free-form English tags. `food_items_f1` goes from 0.665 to 0.861, it seems the adapter tightens which items are extracted without changing the input distribution. `json_valid` and `schema_ok` start high and saturate.

5.3 Qualitative example

This is an example prediction from the test split. It shows the unstructured input, the actual JSON target, and the fine-tuned model's prediction. The baseline fails most reliably on the tags field, it produces a long, free-form English tag list instead of the short codes the dataset uses; the adapter learns to emit the compact codes after fine-tuning.

```
INPUT : A top-down view of a white plate displaying cauliflower florets, a pinch
of baking soda, pieces of rocky road, assorted sushi, sweet potato wedges,
mushroom flatbreads, a drizzle of pesto sauce, jujub ...
```

```
ACTUAL : {"is_food_or_drink": true, "tags": ["fi", "di"], "food_items":
["cauliflower florets", "baking soda", "rocky road", "sushi", "sweet potato
wedges", "mushroom flatbreads", "pesto sauce", "jujube fruits", "raw beef
brisket"], "drink_items": ["white wine"]}
```

```
PRED : {"is_food_or_drink": true, "tags": ["fi", "di"], "food_items":
["cauliflower florets", "baking soda", "rocky road", "sushi", "sweet potato
wedges", "mushroom flatbreads", "pesto sauce", "jujube fruits", "raw beef
brisket"], "drink_items": ["white wine"]}
```

5.4 Visualizations

Four exploratory visualizations appear in the notebook: an input-length histogram (words), a class-label bar chart (food vs not_food), and a tag-vocabulary frequency bar chart in section 1; a prompt+target token-length histogram with the chosen MAX_SEQ_LEN marked in section 2.1. Section 6 reports the training-loss trajectory via the Trainer's logging output. Below: the token-length histogram that justified MAX_SEQ_LEN = 1024 (covers 100% of training rows). Class balance and tag-frequency plots are in the notebook.

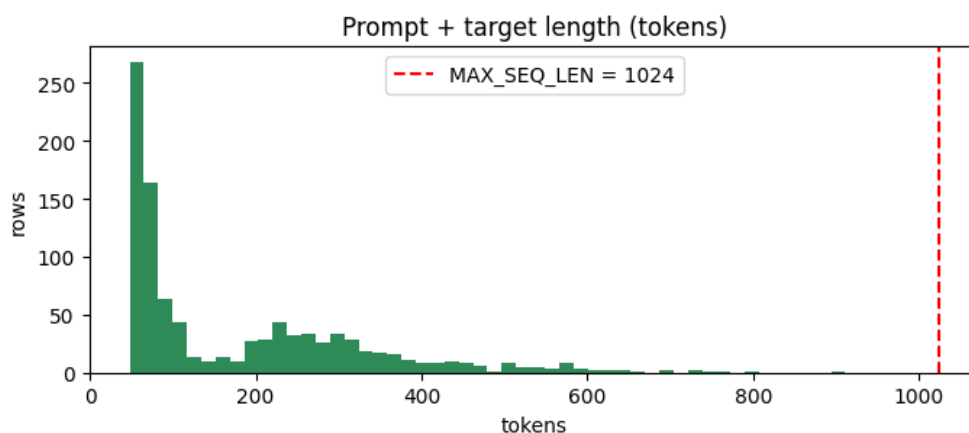


Figure 1. Prompt + target sequence length distribution (train split). The dashed line marks `MAX_SEQ_LEN = 1024`.

6. Discussion

6.1 What worked

The QLoRA recipe is remarkably stable. With the chosen parameters, pipeline runs clean in one pass: 4-bit load on the 3060 Ti, adapter attach, training, save, reload, and finally evaluate. The most interesting result is qualitative: the base model's biggest baseline failure mode, over-generating tags as free-form English phrases instead of the short two-letter codes the dataset uses, is exactly the kind of schema mismatch that an adapter is well suited to fix, even with under a thousand training rows.

6.2 Areas that took longer than expected

The dataset's label column is a Python dict repr, not JSON, and 53 rows out of 1,420 store the `is_food_or_drink` field as a string instead of a bool. It took a while for me to notice them before fixing, they fail silently otherwise. There are sections in the notebook where we clean it up. I was well into the project before I noticed it. The other step that took time for me was deciding what model to use as base model. It took trial-and-error and loading a few different versions and quantizations of Qwen and LLAMA before I decided to use Qwen2.5-3B-Instruct.

6.3 Limitations and extensions

- This run is on a single test split, I'd like to run multiple runs with different seeds and different hyperparameters.
- Small dataset: total rows was 1.2k items, so I'd like to redo this with the larger 135k dataset.
- Tight and specific scope: This model is not generalizable to anything other than creating food output json, and will probably not work with unexpected input (ie, blog posts pulled from the internet)
- Further work: (a) iterate LoRA rank in {4, 8, 16, 32} to find the quality-vs-trainable-params curve, (b) compare 4-bit QLoRA against 8-bit LoRA and full-precision LoRA on the same data, (c) scale the base to Qwen2.5-7B-Instruct under the same 8 GB budget using more aggressive offloading, (d) scale the data to FoodExtract-135k for more extensive training and testing.

References

- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. NeurIPS 2023. arXiv:2305.14314.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. arXiv:2106.09685.
- Qwen Team (2024). Qwen2.5 technical report. arXiv:2412.15115. Model card: <https://huggingface.co/Qwen/Qwen2.5-3B-Instruct>.
- Bourke, D. mrdbourke/FoodExtract-1k dataset. Hugging Face Hub, 2025. <https://huggingface.co/datasets/mrdbourke/FoodExtract-1k>.
- Hugging Face Transformers, PEFT, Datasets, and bitsandbytes — documentation and source code (versions in §4.1).